23. 성능 최적화 및 메트릭 구현 분석

개요

Lighthouse는 고성능 Ethereum 합의 클라이언트로서 다양한 성능 최적화 기법과 포괄적인 메트릭 시스템을 구현하고 있습니다. 이 문서는 Lighthouse의 성능 모니터링 시스템, 메모리 관리 최적화, 그리고 실시간 성능 측정 메커니즘을 상세히 분석합니다.

메트릭 시스템 아키텍처

1. 글로벌 메트릭 레지스트리

Lighthouse는 Prometheus 기반의 중앙집중식 메트릭 시스템을 사용합니다:

```
// common/metrics/src/lib.rs

use prometheus::{Error, HistogramOpts, Opts};

use std::time::Duration;



/// 모든 메트릭을 수집하여 보고

pub fn gather() -> Vec<prometheus::proto::MetricFamily> {

prometheus::gather()

}



/// IntCounter 생성 시도

pub fn try_create_int_counter(name: &str, help: &str) -> Result<IntCounter> {
```

```
let opts = Opts::new(name, help);
let counter = IntCounter::with_opts(opts)?;
prometheus::register(Box::new(counter.clone()))?;
Ok(counter)
}
```

핵심 특징:

- **글로벌 레지스트리**: 모든 메트릭이 하나의 전역 공간에서 관리
- **타입 안전성**: Rust의 타입 시스템을 활용한 메트릭 정의
- **충돌 방지**: 동일한 이름의 메트릭 등록 시 에러 반환

2. 메트릭 타입별 구현

히스토그램 (성능 측정)

```
/// 타이머 시작
pub fn start_timer(histogram: &Result<Histogram>) -> Option<HistogramTimer> {
if let Ok(histogram) = histogram {
Some(histogram.start_timer())
} else {
None
}
}

/// 타이머 종료 및 기간 반환
```

```
pub fn stop_timer_with_duration(timer: Option<HistogramTimer>) -> Duration {
Duration::from_secs_f64(timer.map_or(0.0, |t| t.stop_and_record()))
}
```

카운터 (누적 통계)

```
pub fn inc_counter(counter: &Result<IntCounter>) {
if let Ok(counter) = counter {
counter.inc();
}
}

pub fn inc_counter_by(counter: &Result<IntCounter>, value: u64) {
if let Ok(counter) = counter {
counter.inc_by(value);
}
}
```

게이지 (현재 상태)

```
pub fn set_gauge(gauge: &Result<IntGauge>, value: i64) {
if let Ok(gauge) = gauge {
gauge.set(value);
}
```

```
}
```

성능 모니터링 구현

1. Beacon Chain 성능 메트릭

```
// beacon_node/beacon_chain/src/metrics.rs

pub  static BLOCK_PROCESSING_TIMES: LazyLock<Result<Histogram>> = LazyLock::new(

try_create_histogram(

"beacon_block_processing_seconds",

"Full runtime of block processing",

)

});


pub  static BLOCK_PROCESSING_DB_READ: LazyLock<Result<Histogram>> = LazyLock::ne

try_create_histogram(

"beacon_block_processing_db_read_seconds",

"Time spent loading block and state from DB for block processing",

)

});


pub  static FORK_CHOICE_TIMES: LazyLock<Result<Histogram>> = LazyLock::new(|| {

try_create_histogram_with_buckets(
```

```
"beacon_fork_choice_seconds",
"Full runtime of fork choice",
linear_buckets(10e-3, 20e-3, 10),
)
});
```

주요 성능 지표:

- **블록 처리 시간**: 전체 블록 처리 파이프라인 성능
- **데이터베이스 읽기/쓰기**: I/O 성능 모니터링
- **포크 선택 알고리즘**: 합의 결정 성능
- **상태 전이**: 슬롯 및 에포크 처리 성능

2. 실시간 성능 측정 예시

```
// beacon_node/store/src/hot_cold_store.rs
fn load_hot_state(&self, state_root: Hash256, slot: Slot) -> Result<BeaconState
let _t = metrics::start_timer(&metrics::STORE_BEACON_REPLAY_HOT_BLOCKS_TIME);
// 블록 로딩 및 재생 로직
let blocks = self.load_blocks_to_replay(base_state.slot(), slot, latest_block_r
// 성능 측정이 자동으로 완료됨 (Drop trait 활용)
Ok(final_state)
}
```

3. 검증자 클라이언트 메트릭

```
// validator_client/validator_metrics/src/lib.rs
pub static DUTIES_SERVICE_TIMES: LazyLock<Result<HistogramVec>> = LazyLock::new
try_create_histogram_vec(
"validator_duties_service_times",
"Time taken to perform duties service tasks",
&["task"]
)
});

pub static VC_BEACON_NODE_LATENCY: LazyLock<Result<HistogramVec>> = LazyLock::n
try_create_histogram_vec(
"vc_beacon_node_latency_seconds",
"Latency between validator client and beacon node",
&["beacon_node"]
)
});
```

검증자 성능 지표:

- **듀티 서비스 시간**: 검증자 의무 수행 성능
- **비콘 노드 지연시간**: 네트워크 통신 성능
- **증명 생성 시간**: 서명 및 증명 생성 성능

메모리 관리 최적화

1. 메모리 할당자 최적화

GNU malloc 튜닝

```
// common/malloc_utils/src/glibc.rs
const OPTIMAL_MMAP_THRESHOLD: c_int = 128 * 1_024; // 128KB

pub fn configure_glibc_malloc() -> Result<(), String> {
if !env_var_present(ENV_VAR_MMAP_THRESHOLD) {
if let Err(e) = malloc_mmap_threshold(OPTIMAL_MMAP_THRESHOLD) {
return Err(format!("failed (code {}) to set malloc mmap threshold", e));
}
}
Ok(())
}
```

최적화 전략:

- **mmap 임계값**: 128KB 이상 객체는 mmap 사용
- **힙 단편화 방지**: 동적 임계값 조정 비활성화
- **메모리 사용량 제어**: 힙 크기 안정화

Jemalloc 통합

```
// common/malloc_utils/src/jemalloc.rs
```

```rust
#[global_allocator]
static ALLOC: tikv_jemallocator::Jemalloc = tikv_jemallocator::Jemalloc;


pub fn scrape_jemalloc_metrics() {
epoch::advance().unwrap(); // 통계 업데이트
let num_arenas = arenas::narenas::read().unwrap();
set_gauge(&NUM_ARENAS, num_arenas as i64);
set_gauge(&BYTES_ALLOCATED, stats::allocated::read().unwrap() as i64);
set_gauge(&BYTES_ACTIVE, stats::active::read().unwrap() as i64);
}
```

2. 메모리 사용량 모니터링

```rust
// common/malloc_utils/src/glibc.rs
pub static MALLINFO_ARENA: LazyLock<metrics::Result<IntGauge>> = LazyLock::new(
try_create_int_gauge(
"mallinfo_arena",
"The total amount of memory allocated by means other than mmap(2)",
)
});


pub static MALLINFO_UORDBLKS: LazyLock<metrics::Result<IntGauge>> = LazyLock::n
try_create_int_gauge(
```

```
"mallinfo_uordblks",
"The total number of bytes used by in-use allocations",
)
});

pub fn scrape_mallinfo_metrics() {
let mallinfo = mallinfo();
set_gauge(&MALLINFO_ARENA, unsigned_i64(mallinfo.arena));
set_gauge(&MALLINFO_UORDBLKS, unsigned_i64(mallinfo.uordblks));
// ... 기타 메모리 통계
}
```

시스템 헬스 모니터링

1. 프로세스 및 시스템 메트릭

```
// common/health_metrics/src/metrics.rs
pub static PROCESS_RES_MEM: LazyLock<Result<IntGauge>> = LazyLock::new(|| {
try_create_int_gauge(
"process_resident_memory_bytes",
"Resident memory used by the current process",
)
});
```

```rust
pub static SYSTEM_VIRT_MEM_PERCENTAGE: LazyLock<Result<Gauge>> = LazyLock::new(
try_create_float_gauge(
"system_virt_mem_percentage",
"Percentage of used virtual memory",
)
});


pub static CPU_CORES: LazyLock<Result<IntGauge>> =
LazyLock::new(|| try_create_int_gauge("cpu_cores", "Number of physical cpu cores
```

2. 실시간 시스템 상태 수집

```rust
// common/health_metrics/src/observe.rs
impl Observe for SystemHealth {
#[cfg(target_os = "linux")]
fn observe() -> Result<Self, String> {
let vm = psutil::memory::virtual_memory()
.map_err(|e| format!("Unable to get virtual memory: {:?}", e))?;
let loadavg = psutil::host::loadavg()
.map_err(|e| format!("Unable to get loadavg: {:?}", e))?;
let cpu = psutil::cpu::cpu_times()
.map_err(|e| format!("Unable to get cpu times: {:?}", e))?;
```

```
Ok(Self {
sys_virt_mem_total: vm.total(),
sys_virt_mem_available: vm.available(),
sys_loadavg_1: loadavg.one,
cpu_cores: psutil::cpu::cpu_count_physical(),
// ... 기타 시스템 정보
})
}
}
```

성능 최적화 기법

1. 캐싱 전략

셔플링 캐시

```
// beacon_node/beacon_chain/src/metrics.rs
pub  static SHUFFLING_CACHE_HITS: LazyLock<Result<IntCounter>> = LazyLock::new(|
try_create_int_counter(
"beacon_shuffling_cache_hits_total",
"Count of times shuffling cache fulfils request",
)
});
```

```
pub static SHUFFLING_CACHE_MISSES: LazyLock<Result<IntCounter>> = LazyLock::new
try_create_int_counter(
"beacon_shuffling_cache_misses_total",
"Count of times shuffling cache fulfils request",
)
});
```

상태 캐시 최적화

```
// beacon_node/store/src/state_cache.rs
pub fn get_hdiff_buffer(&mut self, slot: Slot) -> Option<HDiffBuffer> {
if let Some(buffer_ref) = self.hdiff_buffers.get(&slot) {
let _timer = metrics::start_timer_vec(
&metrics::BEACON_HDIFF_BUFFER_CLONE_TIME,
metrics::COLD_METRIC,
);
Some(buffer_ref.clone())
} else {
None
}
}
```

2. 데이터베이스 성능 최적화

압축 및 직렬화

```
// beacon_node/store/src/hot_cold_store.rs
fn store_cold_state(&self, state_root: &Hash256, state: &BeaconState<E>) -> Res
let bytes = state.as_ssz_bytes();
let compressed_value = {
let _timer = metrics::start_timer(&metrics::STORE_BEACON_STATE_FREEZER_COMPRESS
let mut out = Vec::with_capacity(self.config.estimate_compressed_size(bytes.le
let mut encoder = Encoder::new(&mut out, self.config.compression_level)
.map_err(Error::ZstdError)?;
encoder.write_all(&bytes).map_err(Error::ZstdError)?;
encoder.finish().map_err(Error::ZstdError)?;
out
};
self.cold_db.put_bytes(DBColumn::BeaconState, state_root.as_slice(), &compressed
}
```

배치 처리 최적화

```
// beacon_node/store/src/database/redb_impl.rs
pub fn do_atomically(&self, batch: Vec<KeyValueStoreOp>) -> Result<(), Error>
let _timer = metrics::start_timer(&metrics::DISK_DB_WRITE_TIMES);
let write_txn = self.db.begin_write().map_err(Error::DatabaseError)?;
```

```
for op in batch {
match op {
KeyValueStoreOp::PutKeyValue(column, key, value) => {
let table_definition: TableDefinition<'_, &[u8], &[u8]> =
TableDefinition::new(column.into());
let mut table = write_txn.open_table(table_definition)
.map_err(Error::DatabaseError)?;
table.insert(key.as_slice(), value.as_slice())
.map_err(Error::DatabaseError)?;
}
}
}
write_txn.commit().map_err(Error::DatabaseError)?;
metrics::stop_timer(_timer);
Ok(())
}
```

3. 네트워크 성능 최적화

지연시간 측정

```
// validator_client/validator_services/src/latency_service.rs
async fn measure_latency(&self, beacon_node: &BeaconNodeHttpClient) -> Duratio
let start = Instant::now();
```

```
// 간단한 API 호출로 지연시간 측정
let _ = beacon_node.get_node_version().await;
let latency = start.elapsed();
validator_metrics::observe_timer_vec(
&validator_metrics::VC_BEACON_NODE_LATENCY,
&[&beacon_node.id()],
latency,
);
latency
}
```

메트릭 수집 및 내보내기

1. HTTP 메트릭 서버

```
// beacon_node/http_metrics/src/lib.rs
pub fn serve<T: BeaconChainTypes>(
beacon_chain: Arc<BeaconChain<T>>,
config: Config,
) -> Result<(SocketAddr, impl Future<Output = ()>), Error> {
let routes = warp::path("metrics")
.and(warp::get())
.map(move || {
```

```
// 시스템 헬스 메트릭 수집
health_metrics::scrape_health_metrics();
// 메모리 할당자 메트릭 수집
malloc_utils::scrape_allocator_metrics();
// Prometheus 형식으로 메트릭 내보내기
let encoder = TextEncoder::new();
let metric_families = metrics::gather();
let mut buffer = Vec::new();
encoder.encode(&metric_families, &mut buffer).unwrap();
Response::builder()
.header("Content-Type", encoder.format_type())
.body(buffer)
});
let server = warp::serve(routes).bind_with_graceful_shutdown(
config.listen_addr,
async { /* shutdown logic */ }
);
Ok((config.listen_addr, server))
}
```

2. 메트릭 집계 및 분석

버킷 구성 최적화

```rust
// common/metrics/src/lib.rs
pub fn decimal_buckets(min_power: i32, max_power: i32) -> Result<Vec<f64>> {
let mut buckets = Vec::with_capacity(3 * (max_power - min_power + 1) as usize)
for n in min_power..=max_power {
for m in &[1f64, 2f64, 5f64] {
buckets.push(m * 10f64.powi(n))
}
}
Ok(buckets)
}

// 사용 예시: [0.1, 0.2, 0.5, 1.0, 2.0, 5.0, 10.0, 20.0, 50.0]
let buckets = decimal_buckets(-1, 1)?;
```

성능 모니터링 대시보드

1. 핵심 성능 지표 (KPI)

블록 처리 성능:

- `beacon_block_processing_seconds` : 블록 처리 시간
- `beacon_block_processing_db_read_seconds` : DB 읽기 시간
- `beacon_block_processing_state_root_seconds` : 상태 루트 계산 시간

네트워크 성능:

- `vc_beacon_node_latency_seconds` : 비콘 노드 지연시간
- `network_node_bytes_total_received` : 네트워크 수신 바이트
- `network_node_bytes_total_transmit` : 네트워크 송신 바이트

메모리 사용량:

- `process_resident_memory_bytes` : 프로세스 메모리 사용량
- `system_virt_mem_percentage` : 시스템 메모리 사용률
- `mallinfo_uordblks` : 할당된 메모리 크기

2. 알림 및 임계값 설정

```yaml
# Prometheus 알림 규칙 예시
groups:
- name: lighthouse_performance
rules:
- alert: HighBlockProcessingTime
expr: histogram_quantile(0.95, beacon_block_processing_seconds) > 1.0
for: 5m
labels:
severity: warning
annotations:
summary: "블록 처리 시간이 임계값을 초과했습니다"
- alert: HighMemoryUsage
expr: process_resident_memory_bytes / 1024 / 1024 / 1024 > 8
for: 10m
```

```
labels:
severity: critical
annotations:
summary: "메모리 사용량이 8GB를 초과했습니다"
```

성능 튜닝 가이드

1. 메모리 최적화

권장 설정:

```
# Jemalloc 사용 시
export MALLOC_CONF="dirty_decay_ms:1000,muzzy_decay_ms:1000"

# GNU malloc 사용 시
export MALLOC_MMAP_THRESHOLD_=131072 # 128KB
```

2. 데이터베이스 최적화

압축 레벨 조정:

```
// 높은 압축률 vs 빠른 처리 속도 트레이드오프
let compression_level = if high_performance_mode {
1 // 빠른 압축
} else {
```

```
6  // 높은 압축률
};
```

3. 네트워크 최적화

연결 풀 크기 조정:

```
let  http_client = HttpClientBuilder::new()
.pool_max_idle_per_host(10)
.pool_idle_timeout(Duration::from_secs(30))
.build()?;
```

결론

Lighthouse의 성능 최적화 및 메트릭 시스템은 다음과 같은 핵심 특징을 가집니다:

1. **포괄적인 모니터링**: 블록 처리부터 시스템 리소스까지 전방위 모니터링
2. **실시간 성능 측정**: 마이크로초 단위의 정밀한 성능 측정
3. **메모리 최적화**: 할당자 튜닝과 실시간 메모리 사용량 추적
4. **확장 가능한 아키텍처**: Prometheus 기반의 표준화된 메트릭 시스템
5. **자동화된 최적화**: 런타임 성능 데이터를 기반으로 한 동적 최적화

이러한 시스템을 통해 Lighthouse는 대규모 검증자 환경에서도 안정적이고 효율적인 성능을 유지할 수 있습니다.